Guard

Smart contracts security assessment

Final report

Tariff: Standard

BetterBank

November 2025

0xguard.com

hello@0xguard.com

Contents

1. Introduction 3
2. Contracts checked 5
3. Procedure 5
4. Known vulnerabilities checked 6
5. Classification of issue severity 7
6. Issues 7
7. Conclusion 34
8. Disclaimer 35

Introduction

The report has been prepared for **BetterBank**.

The code is available at the @grape-finance/BB-Custom-contracts GitHub repository and was audited after the commit 43fba5ed6a5f47d6ae997660a7fe656c57f44b30.

The scope of the audit is the following smart contracts:

Esteem - an ERC20 token with a single owner and a list of minters.

Favor - an ERC20 token with taxable transfers, a single owner, a list of minters, a restricted function to accrue bonuses in form of ESTEEM tokens.

MintRedeemer - a contract to mint ESTEEM for native EVM coins or allowed payment ERC20 tokens, and to burn ESTEEM to redeem Favor tokens. The contract has a public schedule for ESTEEM rate changes, a single owner can update contract config, including the exchange rates.

Staking - a staking contract for ESTEEM tokens that has 2 privileged accounts that can provide rewards of Favor tokens.

FavorTreasury - a contract that unlocks Favor tokens to the Staking contract as rewards. Unlocking amount is calculated according to circulating supply and token price. A single owner controls list of addresses that is used to calculate the circulating supply.

LPZapper - a contract for user interactions: buy, sell, add liquidity. A single owner controls config of allowed payment tokens and Aave-like pool for flashloans.

MinterOracle - an oracle contract for hardcoded list of supported tokens. Uses either Fetch price feed or TWAP oracles for UniswapV2-like pairs.

UniTWAPOracle - an oracle for UniswapV2-like pairs, uses cumulative prices for TWAP calculations.

LPOracle - an oracle for UniswapV2-like pairs, provides both TWAP prices for tokens of the pair and

USD estimation for LP of the pair, calculated with external oracle for USD prices.

Epoch - an abstract contract to provide updating method with a fixed minimum period.

UsingFetch - a contract in inheritance scheme to provide methods for retrieving data from Fetch price feeds.

Report update. The contracts' code was updated according to this report and rechecked after the commit fba022d617f1c52a9eac30d9802251aecc68375f.

MoneyBin contract is out of scope of this audit.

The updated scope includes 6 new contracts:

LPOracleFairValue - variant of LPOracle with reduced functionality.

MinterOracleV2 - variant of MinterOracle supporting chain of TWAPs.

Killswitch - governance contract that allows approved addresses to enable special condition and requires collegial decision to revert it to original state.

KillswitchPausable - pausing functionality based on Killswitch contract.

EpochKeeper - centralized variant of previous Epoch contract, while the new Epoch is an abstract client for EpochKeeper.

UniswapV2OracleLibrary - modified version of Uniswap library to be used with 0.8.20 version of Solidity.

The LPOracle contract has been removed from the audited repository, all related issues are marked as fixed.

Name	BetterBank
Audit date	2025-09-15 - 2025-11-01

Language	Solidity
Platform	Pulse Chain

Contracts checked

Name	Address
Esteem	
Favor	
MintRedeemer	
Staking	
FavorTreasury	
LPZapper	
MinterOracle	
UniTWAPOracle	
LPOracle	
Epoch	
UsingFetch	
All contracts in scope	
Killswitch	
KillswitchPausable	
LPOracleFairValue	
MinterOracleV2	
EpochKeeper	
UniswapV2OracleLibrary	

Procedure

We perform our audit according to the following procedure:

Automated analysis

- Scanning the project's smart contracts with several publicly available automated Solidity analysis tools
- Manual verification (reject or confirm) all the issues found by the tools

Manual audit

- Manually analyze smart contracts for security vulnerabilities
- Smart contracts' logic check

Known vulnerabilities checked

Title	Check result
Unencrypted Private Data On-Chain	passed
Code With No Effects	passed
Message call with hardcoded gas amount	passed
Typographical Error	passed
DoS With Block Gas Limit	passed
Presence of unused variables	passed
Incorrect Inheritance Order	passed
Requirement Violation	passed
Weak Sources of Randomness from Chain Attributes	passed
Shadowing State Variables	passed
Incorrect Constructor Name	passed
Block values as a proxy for time	passed
Authorization through tx.origin	passed
DoS with Failed Call	passed
Delegatecall to Untrusted Callee	passed

Use of Deprecated Solidity Functions	passed
Assert Violation	passed
State Variable Default Visibility	passed
Reentrancy	passed
Unprotected SELFDESTRUCT Instruction	passed
Unprotected Ether Withdrawal	passed
Unchecked Call Return Value	passed
Floating Pragma	passed
Outdated Compiler Version	passed
Integer Overflow and Underflow	failed
Function Default Visibility	passed

Classification of issue severity

High severity	High severity issues can cause a significant or full loss of funds, change of contract ownership, major interference with contract logic. Such issues require immediate attention.
Medium severity	Medium severity issues do not pose an immediate risk, but can be detrimental to the client's reputation if exploited. Medium severity issues may lead to a contract failure and can be fixed by modifying the contract state or redeployment. Such issues require attention.
Low severity	Low severity issues do not cause significant destruction to the contract's functionality. Such issues are recommended to be taken into consideration.

Issues

High severity issues

1. LP price manipulation (LPOracle)

Status: Fixed

An arbitrary account can influence LP prices returned by the LPOracle by interacting with the LP pair before upcoming call of update function.

Price of LP token is calculated with use of stored `kAverage` variable, which is changed with every update of the oracle. Time-averaging of `kAverage` is performed as sum of stored `lastSqrtK` multiplied by time period from last update to pair's `blockTimestampLast` on the other hand and current `sqrtKNow` multiplied by remaining time from pair's update to `block.timestamp`.

The problem is that time-averaging doesn't increase cost of attack, which can be made in a single-block with flash-loaned funds. In other words, to make oracle return forged price, an actor needs to make 2 sandwich attacks on consecutive oracle update.

```
        // 2) √K TWAP via two-segment integration
        {
            // get reserves + last-reserve-update timestamp
            (uint112 r0, uint112 r1, uint32 tsReserve) = pair.getReserves();
            uint32 nowTs = uint32(block.timestamp);

            // spot √K (Q112.112)
            uint256 sqrtKNow = FixedPoint
                .encode(uint112(Math.sqrt(r0 * r1)))
                .div(uint112(pair.totalSupply()))
                ._x;

            // total time since last oracle update
            uint32 dtFull = nowTs - kTimestampLast;
            require(dtFull > 0, "Oracle: ZERO_TIME");

            // split into before/after reserve-change
            uint32 dt1;
            uint32 dt2;
            if (tsReserve > kTimestampLast) {
```

```
            dt1 = tsReserve - kTimestampLast;
            dt2 = nowTs - tsReserve;
        } else {
            dt1 = dtFull;
            dt2 = 0;
        }

        // patch cumulative: old√K * dt1 + new√K * dt2
        uint256 kC = kCumulativeLast + lastSqrtK * dt1 + sqrtKNow * dt2;

        // compute TWAP over the full window
        kAverage = FixedPoint.uq112x112(
            uint224((kC - kCumulativeLast) / dtFull)
        );

        kCumulativeLast = kC;
        kTimestampLast = nowTs;
        lastSqrtK = sqrtKNow;
    }
```

Recommendation: Consider removing time-averaging and return instant estimation of LP using external USD prices.

Team response: Replaced LPOracle with new fair value reserves formula based off Alpha Homora implementation. Individual token prices are sourced from either 30min TWAP or Fetch price feeds

Update: The LPOracle contract has been removed from the repository and the audit's scope. Check the LPOracleFairValue contract as direct successor.

2. General lack of testing (All contracts in scope)

Status: Fixed

The contracts are not properly tested. We advise covering the core business logic scenarios with integration tests with adequate coverage until we can no longer ensure the project works in line with the documentation, as the audit uncovered several high severity issues and erroneous operation of many methods.

Team response: Test coverage has been improved substantially, now covering 90-100% of all

contracts except minterOracle and MintRedeemer to be added.

3. Owner privileges (All contracts in scope)

Status: Acknowledged

Esteem

The token can be minted by multiple minters. The list of minters is controlled by the contract owner. The contract doesn't provide a direct getter for a full list of authorized minters, which can only be obtained off-chain.

Favor

The token can be minted by multiple minters. The list of minters is controlled by the contract owner. The contract doesn't provide a direct getter for a full list of authorized minters, which can only be obtained off-chain.

The owner can update ESTEEM token address. All pending bonuses will be minted in new token contract.

The owner can set transfer fax up to 50%.

MintRedeemer

The owner can set any price for ESTEEM tokens. Redeeming ESTEEM tokens with unlimited rate results in unlimited mint of Favor tokens.

The owner can update POOL address that is set as a receiver for payments for ESTEEM mint.

The owner can update addresses of the price oracles.

The owner can update list of allowed tokens for payment and for redeeming.

The owner can pause minting and redeeming.

FavorTreasury

The owner can update config addresses: staking, oracle, expansion percentages (within restrictions), list of excluded address, list of trading pairs.

The owner can pause allocating rewards for staking contract.

LPZapper

The owner can update config for supported tokens. Misconfiguration is possible between Favor token and base token, and LP token on the other hand. Another problem can arise if configured base token is not equal to the asset of the POOL contract.

The owner can withdraw locked tokens returned by the router.

The owner can update POOL address to malicious contract or even EOA to collect tokens sent by users.

MinterOracle

The owner can update config addresses to manipulate returned prices.

UniTWAPOracle

The owner can manipulate returned price by setting low `maxPriceCap`.

LPOracle

The owner can update the `masterOracle` address to forge prices.

The owner can manipulate returned price by setting low `maxPriceCap`.

Epoch

The owner can update list of approved addresses.

EpochKeeper

The owner can update epoch duration and, subsequently, `currentEpoch` number. Other epoch-based contracts may become unusable.

Killswitch

The owner can add and remove addresses that can flip engaged condition both ways.

LPOracleFairValue

The owner can update the `masterOracle` address to forge prices.

MinterOracleV2

The owner can update token config to manipulate returned prices.

Recommendation: Secure ownership with timelock and multisig.

Team response: All contracts with owner privileges will be timelocked and set to multisig ownership once protocol is running as intended ~1 week post relaunch. Ownable 2 step also added for extra redundancy.

Medium severity issues

1. Transfers are prohibited (Favor)

Status: Partially fixed

Token transfers are prohibited if destination address contains code unless sender or recipient is in the whitelist. This restriction eliminates transfers between users who use smart accounts as well as using token with DeFi contracts without explicit permission from the project owners.

This issue applies only to the updated code.

Recommendation: Consider removing additional restrictions or add explicit warning.

Team response: Our protocol documentation ensures users are aware that transfers to any contract

whether they be multisig's or smart accounts are not allowed unless we explicitly allow them.

2. Unreliable contract detector (Favor)

Status: Partially fixed

The trading tax is applied if the destination address has code. Transfers to users' multisig accounts or ERC-7702 accounts are subjected to the sell tax up to 50%. Update: transfers are reverted on contract detection.

Non-zero code length is unreliable detector for contracts. It is possible to construct a trading pair that would bypass such detection.

```
    /**
     * @dev See {IERC20-_update}.
     * here we apply taxation.  if destination is a contract, we treat is as a sell.
     * edge cases like buy via approved contracts  while minting  ersteem of whatever
     * will be tax exempt.
     */
    function _update(address _from, address _to, uint256 _value) internal override {
        bool destinationIsContract = _to.code.length != 0;


        if (_isTaxExempt(_to) || _isTaxExempt(_from)) {
            super._update(_from, _to, _value);
            return;
        }

        uint256 taxAmount = 0;

        if (destinationIsContract) {
            taxAmount = (_value * sellTax) / MULTIPLIER;
        }

        // tax goes to treasury
        if (taxAmount > 0) {
            super._update(_from, treasury, taxAmount);
            _value -= taxAmount;
        }
```

```
        super._update(_from, _to, _value);
    }
```

Recommendation: Consider using separate list of affected pairs.

Team response: Favor tx logic changed to revert on contract detection instead of tax. Only designated tax exempt contracts are able to transfer and tax logic will be handled inside these. A user may transfer to a contract in deployment to bypass this but their options after are limited to transfers to EOAs.

3. Circulating supply can be manipulated (FavorTreasury)

Status: Partially fixed

Circulating supply of Favor token is used to calculate rewards to allocate for staking.

Calculations of circulating supply include sum of LP tokens (containing Favor token as one of pair's tokens) held by a list of excluded accounts. The problem is that the pair's reserves can be manipulated in a single transaction. For example, one can swap most of Favor tokens in the pair, then call the `allocateSeigniorage` function, then swap back.

```
     /// @notice Gas intensive if many addresses are added to the list, planned usage
is for ~10 protocol wallets/contracts & 2 LPs at most
    function getFavorCirculatingSupply() public view returns (uint256) {
        uint256 totalSupply = IERC20(favor).totalSupply();

        // Loose Favor balances of excluded addresses
        uint256 balanceExcluded = 0;
        uint256 excludedLen = excludedAddresses.length;
        for (uint256 i = 0; i < excludedLen; i++) {
            balanceExcluded += IERC20(favor).balanceOf(excludedAddresses[i]);
        }

        // Favor in LP tokens held by excluded addresses
        uint256 favorFromLpHeldByExcluded = 0;
        uint256 pairsLen = lpPairsToExclude.length;

        // Loop through LP pairs excluded and calculate Favor reserves in each
```

```
        for (uint256 j = 0; j < pairsLen; j++) {
            address pair = lpPairsToExclude[j];
            if (!isLpPairToExclude[pair]) continue;

            IUniswapV2Pair p = IUniswapV2Pair(pair);

            (uint112 r0, uint112 r1, ) = p.getReserves();
            address t0 = p.token0();
            uint256 favorReserves = (t0 == favor) ? uint256(r0) : uint256(r1);

            uint256 lpTotal = p.totalSupply();
            if (lpTotal == 0 || favorReserves == 0) continue;

            // Get excluded LP token balance of each excluded address and add to
excluded balance of Favor
            for (uint256 i = 0; i < excludedLen; i++) {
                uint256 holderLp = p.balanceOf(excludedAddresses[i]);
                if (holderLp == 0) continue;
                favorFromLpHeldByExcluded += (favorReserves * holderLp) / lpTotal;
            }
        }

        return totalSupply - balanceExcluded - favorFromLpHeldByExcluded;
    }
```

Recommendation: LP tokens should be excluded from calculation of the circulating supply.

Team response: LP tokens excluded include only favor/favor LPs that only the protocol owns that we use for liquidation routing. Users can trade this however the revert on transfer to contract on the pair would mean they would be stuck with the token and would need to sell through the normal LP to base token for a 50% tax negating any possible profit, user also needs to acquire Esteem legitimately via minting to stake to earn new Favor.

4. Incorrect compiler version (UniTWAPOracle)

Status: Fixed

Incorrect pragma is used, UniswapV2OracleLibrary math requires unchecked math, but Solidity 0.8.20 is used.

```
pragma solidity >=0.5.0;

import '@uniswap/v2-core/contracts/interfaces/IUniswapV2Pair.sol';
import '@uniswap/lib/contracts/libraries/FixedPoint.sol';

// library with helper methods for oracles that are concerned with computing average
prices
library UniswapV2OracleLibrary {
    using FixedPoint for *;

    // helper function that returns the current block timestamp within the range of
uint32, i.e. [0, 2**32 - 1]
    function currentBlockTimestamp() internal view returns (uint32) {
        return uint32(block.timestamp % 2 ** 32);
    }

    // produces the cumulative price using counterfactuals to save gas and avoid a call
to sync.
    function currentCumulativePrices(
        address pair
    ) internal view returns (uint price0Cumulative, uint price1Cumulative, uint32
blockTimestamp) {
        blockTimestamp = currentBlockTimestamp();
        price0Cumulative = IUniswapV2Pair(pair).price0CumulativeLast();
        price1Cumulative = IUniswapV2Pair(pair).price1CumulativeLast();

        // if time has elapsed since the last update on the pair, mock the accumulated
price values
        (uint112 reserve0, uint112 reserve1, uint32 blockTimestampLast) =
IUniswapV2Pair(pair).getReserves();
        if (blockTimestampLast != blockTimestamp) {
            // subtraction overflow is desired
            uint32 timeElapsed = blockTimestamp - blockTimestampLast;
            // addition overflow is desired
            // counterfactual
            price0Cumulative += uint(FixedPoint.fraction(reserve1, reserve0)._x) *
timeElapsed;
            // counterfactual
            price1Cumulative += uint(FixedPoint.fraction(reserve0, reserve1)._x) *
timeElapsed;
        }
```

```
    }
}
```

Recommendation: Use pre-0.8 version or rewrite the UniswapV2OracleLibrary.

5. Possible incorrect or stale price (UniTWAPOracle)

Status: Acknowledged

The oracle returns price as step function of average prices. The oracle may return stale prices if called via `consult` function. Update period can't be set lower than 15 minutes.

The oracle returns capped price that can be significantly different from fair price.

```
    /// @notice Will always return 0 before update has been called successfully for the
first time.
    function consult(
        address _token,
        uint256 _amountIn
    ) external view returns (uint256 amountOut) {
        if (_token == token0) {
            amountOut = uint256(price0Average.mul(_amountIn).decode144()); // Uses
FixedPoint library for mul of uq112x112
        } else {
            require(_token == token1, "Oracle: INVALID_TOKEN");
            amountOut = uint256(price1Average.mul(_amountIn).decode144());
        }

        if (maxPriceCap > 0 && amountOut > maxPriceCap) {
            amountOut = maxPriceCap;
        }
    }
```

The only way to ensure price freshness is to perform an external check of `blockTimestampLast`.

Recommendation: Include freshness check or add documentation with warnings.

Team response: Price cap in oracle contract is intended and documented in protocol docs.

6. Incorrect estimation of LP price (LPOracle)

Status: Fixed

Calculation of `sqrt(K)/totalSupply` doesn't include fee part that is applied to every pair's liquidity change if `pair.feeTo()` is set. Correct implementation can be found in `UniswapV2LiquidityMathLibrary.getLiquidityValue`.

Team response: LP oracle changed to use Fair value calculation (LPOracleFairValue.sol) , LPOracle.sol no longer is being used.

Update: The LPOracle contract has been removed from the repository and the audit's scope. Check the LPOracleFairValue contract as direct successor.

7. Possible incorrect or stale price (LPOracle)

Status: Fixed

The oracle returns price as step function of average prices. The oracle may return stale prices if called via `consult` function. Update period can't be set lower than 15 minutes.

The oracle returns capped price that can be significantly different from fair price.

```
    /// @notice Update TWAPs: Uniswap prices, √K, and USD feeds
    function update() external onlyApproved checkEpoch {
        ...
            if (lastLpPrice == 0 || lpPriceCap == 0) {
                lastLpPrice = newLpPrice;
            } else {
                uint256 maxAllowed = (lastLpPrice * lpPriceCap) / 1e18;
                if (newLpPrice <= maxAllowed) {
                    lastLpPrice = newLpPrice;
                }
            }
    }

    /// @notice Consult TWAP: USD per unit of token0, token1 (18-decimals) or LP
    /// @dev Deployer must call update() first after deployment or this will always
return 0
    function consult(
```

```
        address _token,
        uint256 _amountIn
    ) external view returns (uint256 amountOut) {
        if (_token == token0) {
            amountOut = uint256(price0Average.mul(_amountIn).decode144());
        } else if (_token == token1) {
            amountOut = uint256(price1Average.mul(_amountIn).decode144());
        } else if (_token == address(pair)) {
            amountOut = lastLpPrice;
        } else {
            revert("Oracle: INVALID_TOKEN");
        }
    }
```

Recommendation: Include freshness check or add documentation with warnings.

Team response: LP oracle changed to use Fair value calculation (LPOracleFairValue.sol) , LPOracle.sol no longer is being used.

Update: The LPOracle contract has been removed from the repository and the audit's scope. Check the LPOracleFairValue contract as direct successor.

8. Incorrect compiler version (LPOracle)

Status: Fixed

Incorrect pragma is used, UniswapV2OracleLibrary math requires unchecked math, but Solidity 0.8.20 is used.

Calculation of cumulative variables requires overflowing, but `kCumulativeLast` is calculated with safe math addition.

```
pragma solidity >=0.5.0;

library UniswapV2OracleLibrary { ... }

contract LPOracle is Epoch {
    function update() external onlyApproved checkEpoch {
        ...
```

```
            uint256 kC = kCumulativeLast + lastSqrtK * dt1 + sqrtKNow * dt2;

            uint256 newUsd0C = usd0CumulativeLast + u0 * dtU;
            uint256 newUsd1C = usd1CumulativeLast + u1 * dtU;
        }
    }
```

Recommendation: Use pre-0.8 version or rewrite the UniswapV2OracleLibrary and calculation of cumulative variables.

Update: The LPOracle contract has been removed from the repository and the audit's scope. Check the LPOracleFairValue contract as direct successor.

9. Epoch period can be updated retrospectively (Epoch)

Status: Fixed

Function `setPeriod` allows owner to update period variable that affects `nextEpochPoint` function and `checkEpoch` modifier. For example, if there're 10 pending epochs with 100 minutes period, then the owner can instantly change this number to 5 or 20.

Recommendation: `setPeriod` should update epoch counter before changing the period.

Team response: Epoch system updated with global keeper based on block.timestamp for relevant contracts and abstract epoch for individual deployments of FavorTreasury and TWAP oracle. Protocol intended epoch is 1hr only in a system revamp would this be adjusted differently.

10. Incorrect estimation of LP price (LPOracleFairValue)

Status: Acknowledged

Pair's total supply may return inaccurate data if `feeOn` is set and pair's `K` is not updated. Correct implementation can be found in `UniswapV2LiquidityMathLibrary.getLiquidityValue`.

Team response: LP will remain underpriced which is acceptable and desired for its use case as collateral in lending. The system by design encourages frequent LP adds/removes through new positions and frequent liquidations which in turn keeps total supply from feeOn close to actual.

11. Possible stale price (MinterOracleV2)

Status: Fixed

Token config includes maxAge as staleness guard and the function getLatestPrice is declared as reverting on stale price. But price is checked for staleness only for tokens with Fetch feeds, tokens to be obtained from TWAP oracles are not checked.

Recommendation: Include freshness check or add documentation with warnings.

Low severity issues

1. Locking native currency (MintRedeemer)

Status: Fixed

The contract accepts native currency without effect. Only the owner can recover locked ETH.

```
receive() external payable {}
```

Recommendation: Remove empty `receive` fucntion.

2. Limited token support (MintRedeemer)

Status: Acknowledged

Payment tokens with decimals greater than 18 are not supported. Function `_normalizeTokenAmount` reverts if called with unsupported token.

```
    // Standardize all tokens to 18 decimals
    function _normalizeTokenAmount(uint256 amount, address token) internal view returns
(uint256) {

        uint8 tokenDecimals = token == address(0)
                ? 18
                : IERC20Metadata(token).decimals();

        if (tokenDecimals == 18) {
            return amount;
        } else {
```

```
            // e.g. USDT: 6 → bump 10**(18-6)
            return amount * (10 ** (18 - tokenDecimals));
        }
    }
```

Team response: We don't intend to list any tokens with > 18 decimals at any time in the future.

3. Limited history length (Staking)

Status: Acknowledged

Reward history is stored in circular buffer of length MAX_HISTORY. If users haven't updated their state for a long period that covers the history buffer, then part of their rewards will be deleted.

```
    function allocateSeigniorage(uint256 amount) external nonReentrant whenNotPaused {
        require(msg.sender == owner() || msg.sender == treasuryOperator, "Not
authorized");
        require(amount > 0, "Grove: Cannot allocate 0");
        require(totalSupply() > 0, "Grove: Cannot allocate when totalSupply is 0");

        // Create & add new snapshot
        uint256 prevRPS = getLatestSnapshot().rewardPerShare;
        uint256 nextRPS = prevRPS + ((amount * 1e18) / totalSupply());

        historyEnd += 1;
        groveHistory[historyEnd] = GroveSnapshot({ time: block.timestamp,
rewardReceived: amount, rewardPerShare: nextRPS });

        if (historyEnd - historyStart + 1 > MAX_HISTORY) {
            delete groveHistory[historyStart];
            historyStart += 1;
        }

        favor.safeTransferFrom(msg.sender, address(this), amount);
        emit RewardAdded(msg.sender, amount);
    }
```

Team response: New history limit has been reset to 50,000 with an intended epoch duration of 1hr this gives a maxhistory of 5.7yrs. At that point if a user hasn't interacted with the contract in anyway

we will count their rewards as null in benefit to active users of the system.

4. Incorrect getter function (Staking)
Status: Acknowledged

The function `isLocked` may be unusable via block explorer since it does not receive user address in parameters.

This issue applies only to the updated code.

```
    //  whether caller stake is locked
    function isLocked() public view returns (bool) {
        return epochKeeper.currentEpoch() <= stakeLock[_msgSender()];
    }
```

Recommendation: Consider to use address from function's parameters instead of msg.sender.

Team response: `isLocked()` is a helper for withdraw function to check msg.sender's current lock. `stakeLock` mapping is public and therefore has a default getter function taking in a user's address which will return that address's lock epoch.

5. Gas limit problem (FavorTreasury)
Status: Acknowledged

Possible unlimited gas usage when searching over the arrays of addresses. Affected functions: `removeExcludedAddress, removeLpPairToExclude, getFavorCirculatingSupply`.

Team response: Maximum intended usage of these arrays is 5-10 protocol owned/controlled addresses.

6. Gas limit problem (LPZapper)
Status: Acknowledged

Possible unlimited gas usage when searching over the arrays of addresses. Affected functions: `zapToken, zapPLS`.

Team response: Intended Dust Tokens array is 20-50 tokens maximal.

7. Possible lock of tokens (LPZapper)

Status: Fixed

`zapToken` or `zapPLS` functions swap half of the user's funds for the Favor tokens, then add liquidity with use of other half. If the zapped amount is significant comparing to the pair's reserves, the first swap can shift the pair's ratio, and the later liquidity increase will result in remaining tokens being returned to the LPZapper contract. If some of supported tokens aren't in the `dustTokens` list, they'll be locked unless the owner calls `adminWithdraw` or `adminWithdrawPLS`.

Team response: We have modified the addFavor function in the zapper to auto add the favor and base token as dust tokens and only these tokens can be used in the zap and swaps.

8. Oracle depends on DAI price (MinterOracle)

Status: Acknowledged

The oracle crucially relies on Fetch price feeds to not deviate from on-chain data. `getPlsSpotPrice` and `getDaiSpotPrice` are sensitive to DAI price and revert in DAI depeg scenario.

This issue applies only to the updated code.

```
    function getDaiSpotPrice() public view returns (uint256) {
        bytes memory _queryData = abi.encode(
            "SpotPrice",
            abi.encode("dai", "usd")
        );
        bytes32 _queryId = keccak256(_queryData);

        (bytes memory _value, uint256 _timestampRetrieved) = getDataBefore(
            _queryId,
            block.timestamp - 20 minutes
        );
        if (_timestampRetrieved == 0) return 0;
        require(
            block.timestamp - _timestampRetrieved < 24 hours,
            "Data timestamp is more than 24 hours."
        );
        uint256 fetchPrice = abi.decode(_value, (uint256));
```

```
        // Bounds: 0.98 .. 1.02
        uint256 lower = (ONE * (10_000 - TWO_PERCENT)) / 10_000;
        uint256 upper = (ONE * (10_000 + TWO_PERCENT)) / 10_000;

        require(
            fetchPrice >= lower && fetchPrice <= upper,
            "Dai peg off by >2%"
        );

        return fetchPrice;
    }
```

Team response: This is intended functionality to halt mints/redeems with dai during the event of a 2% + depeg on chain.

9. Oracle depends on pair reserves (MinterOracle)

Status: Acknowledged

The `getPlsxSpotPrice` and `getPlsSpotPrice` functions use instant pair's reserves to validate external Fetch price. Uniswap pair can be manipulated by anyone to cause oracle revert (DoS attack).

```
    function getPlsSpotPrice() public view returns (uint256) {
        bytes memory _queryData = abi.encode(
            "SpotPrice",
            abi.encode("pls", "usd")
        );
        bytes32 _queryId = keccak256(_queryData);

        (bytes memory _value, uint256 _timestampRetrieved) = getDataBefore(
            _queryId,
            block.timestamp - 20 minutes
        );
        if (_timestampRetrieved == 0) return 0;
        require(
            block.timestamp - _timestampRetrieved < 24 hours,
            "Data timestamp is more than 24 hours."
        );

        uint256 fetchPrice = abi.decode(_value, (uint256));
```

```
        address[] memory path = new address[](2);
        path[0] = wpls;
        path[1] = dai;
        uint256 dexPrice = _dexPrice(path);

        uint256 diff = _absDiff(fetchPrice, dexPrice);
        require(
            diff * 10_000 <= dexPrice * maxBps,
            "Dex guard: deviation too large"
        );

        return fetchPrice;
    }

    function _dexPrice(
        address[] memory path
    ) internal view returns (uint256 dexPrice) {
        require(path.length >= 2, "Bad path");

        uint[] memory amts;
        try router.getAmountsOut(ONE, path) returns (uint[] memory r) {
            amts = r;
        } catch {
            revert("No DEX route/liquidity");
        }

        uint256 outRaw = amts[amts.length - 1];

        uint8 outDec = IERC20Metadata(path[path.length - 1]).decimals();
        if (outDec >= 18) {
            dexPrice = outRaw / (10 ** (outDec - 18));
        } else {
            dexPrice = outRaw * (10 ** (18 - outDec));
        }
    }
```

Recommendation: Data from pair should be used only for informational purposes, e.g., emitting an event.

Team response: We acknowledge the risk of DDOS and manage this via only using the highest liquidity pair on chain which makes the ongoing cost of an attack very high.

10. Possible manipulated price (UniTWAPOracle)

Status: Acknowledged

Function `twap` returns average price for the time period since last update. Short observation window can be prone to price manipulation.

```
    /// @notice Returns rolling TWAP price which is more likely to be prone to short
term price fluctuations. Only used on UI for informational purposes.
    function twap(address _token, uint256 _amountIn) external view returns (uint256
_amountOut) {
        (uint256 price0Cumulative, uint256 price1Cumulative, uint32 blockTimestamp) =
            UniswapV2OracleLibrary.currentCumulativePrices(address(pair));

        uint32 timeElapsed;
        unchecked { timeElapsed = blockTimestamp - blockTimestampLast; } // Overflow
desired wrapped in unchecked
        if (timeElapsed == 0) return 0;

        if (_token == token0) {
            uint256 price0Delta;
            unchecked { price0Delta = price0Cumulative - price0CumulativeLast; }
            _amountOut = uint256(
                FixedPoint.uq112x112(uint224(price0Delta /
timeElapsed)).mul(_amountIn).decode144()
            );
        } else if (_token == token1) {
            uint256 price1Delta;
            unchecked { price1Delta = price1Cumulative - price1CumulativeLast; }
            _amountOut = uint256(
                FixedPoint.uq112x112(uint224(price1Delta /
timeElapsed)).mul(_amountIn).decode144()
            );
        } else {
            revert("Oracle: INVALID_TOKEN");
        }
```

```
        if (maxPriceCap > 0 && _amountOut > maxPriceCap) {
            _amountOut = maxPriceCap;
        }
    }
```

Team response: TWAP function in Oracle is only intended for informational purposes as natspec notice comments.

11. Unsafe functions (LPOracle)

Status: Fixed

Functions `redeemableUsdPerLpQ112` and `redeemableUsdPerLpScaled` are based on the onchain data, they return unreliable prices that can be manipulated.

```
    /// @notice Raw redeemable USD per LP in Q112.112
    function redeemableUsdPerLpQ112() public view returns (uint256) {
        (uint112 r0, uint112 r1, ) = pair.getReserves();
        uint256 px0 = uint256(usd0Average._x);
        uint256 px1 = uint256(usd1Average._x);
        uint256 totalUsdQ = uint256(r0) * px0 + uint256(r1) * px1;
        return totalUsdQ / pair.totalSupply();
    }

    /// @notice Redeemable USD per LP scaled by 1e18
    function redeemableUsdPerLpScaled() public view returns (uint144) {
        uint256 q112 = redeemableUsdPerLpQ112();
        uint256 scaled = (q112 * 1e18) >> 112;
        return uint144(scaled);
    }
```

Recommendation: Remove unsafe functions or add documentation with warnings.

Team response: Redeemable price is copied over in new LPOracleFairValue.sol however is only used for sanity check spot price not used in calculations. Natspec notiice is added to assert this.

Update: The LPOracle contract has been removed from the repository and the audit's scope. Check the LPOracleFairValue contract as direct successor.

12. Misleading calculations (LPOracle)

Status: Fixed

The contract tracks USD prices from external source and their cumulative sums over time.

In fact, `usd0CumulativeLast` and `usd1CumulativeLast` aren't used and could be safely omitted, as `usd0Average` is calculated only from `masterOracle.getLatestPrice(token0)`.

```
        // 3) USD per-token TWAPs (custom feed)
        {
            uint32 nowTsU = uint32(block.timestamp);
            uint256 u0 = masterOracle.getLatestPrice(token0);
            uint256 u1 = masterOracle.getLatestPrice(token1);
            uint32 dtU = nowTsU - usdTimestampLast;
            require(dtU > 0, "Oracle: ZERO_TIME");

            uint256 newUsd0C = usd0CumulativeLast + u0 * dtU;
            uint256 newUsd1C = usd1CumulativeLast + u1 * dtU;

            uint256 avg0Raw = (newUsd0C - usd0CumulativeLast) / dtU;
            uint256 avg1Raw = (newUsd1C - usd1CumulativeLast) / dtU;

            uint256 avg0Q112 = (avg0Raw << 112) / 1e18;
            uint256 avg1Q112 = (avg1Raw << 112) / 1e18;

            usd0Average = FixedPoint.uq112x112(uint224(avg0Q112));
            usd1Average = FixedPoint.uq112x112(uint224(avg1Q112));

            usd0CumulativeLast = newUsd0C;
            usd1CumulativeLast = newUsd1C;
            usdTimestampLast = nowTsU;
        }
```

Team response: LPOracle.sol contract made redundant in favor of LPOracleFairValue.sol contract.

Update: The LPOracle contract has been removed from the repository and the audit's scope. Check the LPOracleFairValue contract as direct successor.

13. No getter for approved users (Epoch)

Status: Fixed

Total list of authorized accounts is not available on-chain.

```
    mapping(address => bool) public isApprovedUser;

    modifier onlyApproved() {
        require(
            isApprovedUser[msg.sender] || msg.sender == owner(),
            "Epoch: caller not approved"
        );
        _;
    }
```

Recommendation: Use EnumerableSet to store list of approved users.

Team response: Epoch abstract now has a public approved user mapping.

Comment on reponse: Public mapping doesn't provide full list of approved users. Full list can only be constructed off-chain via events.

14. Unsafe functions (LPOracleFairValue)

Status: Partially fixed

Functions `redeemableUsdPerLpQ112` and `redeemableUsdPerLpScaled` are based on the onchain data, they return unreliable prices that can be manipulated.

```
    /// @notice Raw redeemable USD per LP in Q112.112 (arithmetic sum of reserves), for
sanity check spot price not used in calculations
    function redeemableUsdPerLpQ112(
        address pair
    ) public view returns (uint256) {
        IUniswapV2Pair p = IUniswapV2Pair(pair);
        (uint112 r0, uint112 r1, ) = p.getReserves();
        if (r0 == 0 || r1 == 0) return 0;

        address token0 = p.token0();
```

```
        address token1 = p.token1();

        uint256 px0 = getUSDPx_Q112(token0);
        uint256 px1 = getUSDPx_Q112(token1);
        if (px0 == 0 || px1 == 0) return 0;

        uint256 value0Q112 = Math.mulDiv(uint256(r0), px0, 1e18);
        uint256 value1Q112 = Math.mulDiv(uint256(r1), px1, 1e18);
        uint256 totalUsdQ112 = value0Q112 + value1Q112;

        // Return Q112 value
        return Math.mulDiv(totalUsdQ112, 1e18, p.totalSupply());
    }

    /// @notice Redeemable USD per LP in WAD (1e18), for sanity check spot price not
used in calculations
    function redeemableUsdPerLpScaled(
        address pair
    ) public view returns (uint144) {
        uint256 q112 = redeemableUsdPerLpQ112(pair);
        uint256 scaled = Math.mulDiv(q112, 1e18, 1 << 112);
        return uint144(scaled);
    }
```

Recommendation: Remove unsafe functions or add documentation with warnings.

Team comment: Redeemable price is copied over in new LPOracleFairValue.sol however is only used for sanity check spot price not used in calculations. Natspec notiice is added to assert this.

15. Oracle depends on pair reserves (MinterOracleV2)

Status: Acknowledged

The `getLatestPrice` function uses instant pair's reserves to validate external Fetch price. Uniswap pair can be manipulated by anyone to cause oracle revert (DoS attack).

```
    /// @notice Returns USD price with 1e18 decimals; reverts if not configured or
stale.
    function getLatestPrice(address token) public view returns (uint256) {
        if (token == address(0)) token = WPLS;
```

```
        return _priceUSD(token, 0);
    }

    function _priceUSD(
        address token,
        uint256 depth
    ) internal view returns (uint256) {
        (. . .)
        _enforceDexGuard(token, price);
        return price;
    }

    function _enforceDexGuard(
        address token,
        uint256 usdPrice1e18
    ) internal view {
        DexGuard storage g = dexGuards[token];
        if (!g.enabled) return;

        uint256 dexPx = _dexUsdPrice(g.path);
        uint256 diff = (usdPrice1e18 > dexPx)
            ? (usdPrice1e18 - dexPx)
            : (dexPx - usdPrice1e18);

        require(
            diff * 10_000 <= dexPx * g.maxBps,
            "Dex guard: deviation too large"
        );
    }

    function _dexUsdPrice(
        address[] storage path
    ) internal view returns (uint256 px1e18) {
        require(path.length >= 2, "bad path");

        uint8 inDec = IERC20Metadata(path[0]).decimals();
        uint256 amountIn = (inDec >= 18)
            ? (ONE * (10 ** (inDec - 18)))
            : (ONE / (10 ** (18 - inDec)));

        uint[] memory amts = router.getAmountsOut(amountIn, _toMem(path));
```

```
        uint256 outRaw = amts[amts.length - 1];

        uint8 outDec = IERC20Metadata(path[path.length - 1]).decimals();
        px1e18 = (outDec >= 18)
            ? (outRaw / (10 ** (outDec - 18)))
            : (outRaw * (10 ** (18 - outDec)));
    }
```

Recommendation: Data from pair should be used only for informational purposes, e.g., emitting an event.

Team response: We acknowledge the risk of DDOS and manage this via only using the highest liquidity pair on chain which makes the ongoing cost of an attack very high.

Conclusion

BetterBank Esteem, Favor, MintRedeemer, Staking, FavorTreasury, LPZapper, MinterOracle, UniTWAPOracle, LPOracle, Epoch, UsingFetch, All contracts in scope, Killswitch, KillswitchPausable, LPOracleFairValue, MinterOracleV2, EpochKeeper, UniswapV2OracleLibrary contracts were audited.
3 high, 11 medium, 15 low severity issues were found.
2 high, 6 medium, 5 low severity issues have been fixed in the update.

Privileged roles have extensive control over the protocol's core functions and funds. To address this, we recommend implementing robust security practices, including the use of multi-signature wallets, timelocks, and hardware wallets for privileged accounts management. Users are required to trust that the administrators that these accounts are properly secured and will not act maliciously.

Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability)set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without 0xGuard prior written consent.

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts 0xGuard to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.



Guard